

DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback

Yunpeng Dong¹, Jingkai He^{1,2}, Yuze Hou¹, Dong Du^{1,2}, Zhonghu Xu³, Si Yu³, Yubin Xia^{1,2},
Haibo Chen^{1,2}

¹Institute of Parallel and Distributed Systems, Shanghai Jiao Tong University

²Engineering Research Center for Domain-specific Operating Systems, Ministry of Education, China

³Huawei Technologies Co., Ltd.

Abstract

LLM-powered AI agents require high-frequency state exploration (e.g., test-time tree search and reinforcement learning), relying on rapid checkpoint and rollback (C/R) of the complete sandbox state, including files and process state (e.g., memory, contexts, etc.). Existing mechanisms duplicate the entire state, causing hundreds of milliseconds to seconds of latency per C/R, which severely bottlenecks deep search and large-scale fan-outs.

This paper observes that subsequent checkpoints in AI agents are highly similar. Therefore, instead of full duplication, a sandbox should *only duplicate the changes between consecutive checkpoints* (**Key Insight**). However, it is non-trivial to realize the idea, mainly due to the missing OS supports. This paper proposes a new OS-level abstraction, *DeltaState*, to enable the change-based transactional C/R for AI agents with two co-designed OS mechanisms. First, *DeltaFS* enables change-based filesystem C/R by organizing the file states into layers and dynamically freezing the writable layer and inserting a new one during checkpoint, reducing file updates to copy-on-write, and making rollback a simple layer switch. Second, *DeltaCR* enables change-based process state C/R using incremental dumps, and accelerates rollback by bypassing traditional pipelines to directly fork() from a frozen template process. We then present DeltaBox, a novel agent sandbox achieving millisecond level C/R through the two new mechanisms. Evaluations on SWE-bench and RL micro-benchmarks show DeltaBox completes checkpoint and rollback in millisecond-level latency (14ms and 5ms, respectively), empowering agents to explore substantially more nodes under fixed time budgets.

Keywords: AI Agent, Sandbox, Checkpoint/Restore, Overlays, CRIU, Copy-on-Write, State Management

1 Introduction

LLM-powered AI agents have emerged as a primary approach to automating complex software engineering tasks. From automated code repair [1] to web navigation [2], these agents operate by iteratively generating actions [3],

executing them within a secure sandboxed environment [4], observing outcomes, and refining strategies. A recent trend to enhance agent capabilities is test-time compute scaling [5]. State-of-the-art agents invest additional inference compute through systematic tree search (e.g., Monte Carlo Tree Search [6] or Best-of-N sampling), exploring multiple candidate paths, verifying them against real execution feedback, and backtracking from failures.

However, applying tree search to stateful OS-level environments creates a severe infrastructure bottleneck. In early text-based tasks (e.g., HotPotQA), rollback was (almost) cost-free, achieved by simply truncating prompt history. Conversely, for more complex agent tasks [7, 8], agent actions may produce irreversible side effects. E.g., for a coding agent: `rm` deletes files, `pip install` mutates the package tree, and `sed` rewrites source code. Both *rollback* and *fork* therefore require rapid checkpoint and rollback (C/R)¹ of the complete sandbox state.

The sandbox state encompasses two tightly coupled dimensions: durable filesystem state (e.g., working directories, installed packages) and ephemeral process state (e.g., process memory, python interpreter heap, open file descriptors, etc.). These dimensions must be captured and restored jointly to prevent state divergence or context loss.

Fast C/R becomes even more pressing with modern reasoning models (o1-class [9], DeepSeek-R1 [10]), which internalize search within extended chains of thought but still execute code at each reasoning step via tool-use variants, trying alternative dependency installations, running test suites, and reverting failing patches. Execution-guided sampling allocates inference budget across many parallel candidate trajectories [5, 11], each often requiring an isolated sandbox clone. Within each trajectory, iterative debug-test loops (generate patch → run tests → analyze failure → revise → repeat) demand fine-grained intermediate checkpoints. This creates a two-dimensional scaling challenge: horizontal scaling (many parallel trajectories, each needing a fast initial

 Corresponding author: Dong Du (dd_nirvana@sjtu.edu.cn).

¹We use rollback and restore interchangeably in the paper. While synonymous, rollback implies the agent’s viewpoint, whereas restore refers to the underlying sandbox mechanism.

clone) and vertical depth (each trajectory’s internal search tree, requiring checkpoint/restore of intermediate states).

Beyond test-time inference, training agent policies with reinforcement learning (RL) [12–14] has emerged as a parallel infrastructure pressure point. Each RL training step issues a batch of k rollouts against a sandbox, scores them with a reward model, and updates the policy; production systems must repeatedly tear down and re-create k *independent* sandboxes per training step, each at a known-good warm starting state (the same testbed snapshot, with the same toolchain pre-loaded). Today’s deployed approaches rebuild this warm state by either committing a Docker layer per starting state and running a fresh container per rollout (latency dominated by container start and image pull), or by snapshotting and resuming a microVM per rollout (latency dominated by guest memory pre-touch and device re-attach). Both sit in the hundreds-of-milliseconds to seconds regime per fork; with k in the tens to low hundreds, fork latency directly bounds training throughput.

Existing systems manage the filesystem and process memory in isolation, typically achieving C/R by duplicating the entire state into a checkpoint image. While full duplication works well for scenarios like serverless computing that prioritize cold-start restores [15–18], it is prohibitively slow for AI agents requiring high-frequency checkpoints on the critical path. Prior efforts face three main challenges:

Challenge-1: High latency of agent checkpointing. Prior sandboxes usually focuses on optimizations for restore but not checkpoint, e.g., in serverless computing [16, 18, 19, 19–24], checkpointing is happened offline which is usually one-time cost, while restore directly impacts the cold start latency. As a result, current agent sandbox systems suffer high latency of checkpointing. E.g., E2B takes 4 seconds to pause a sandbox per 1 GiB of RAM [25]. Other approaches, e.g., CRIU [26] take seconds for multi-GiB process footprints. Docker commits take several seconds for non-trivial layer changes, and VM-level snapshots (e.g., Firecracker [27]) incur hundreds of milliseconds to seconds of latency [15]. These latencies severely bottleneck deep search and large-scale RL fan-outs.

Challenge-2: Lack of efficient C/R for durable file states. A significant challenge is how to efficiently checkpoint and restore the filesystem state of an agent sandbox. Current systems usually adopt file copying, which incurs high latency. Methods like docker commits, git stash/branch, and VM-level snapshots [27] all slow in such cases.

This paper observes that subsequent checkpoints in AI agent workloads are highly similar, with only minor incremental changes between steps (e.g., a few new files or modified memory pages). Therefore, we argue that instead of duplicating the entire state, a sandbox should *only duplicate the changes between consecutive checkpoints* (**Key Insight**).

To this end, we present DeltaBox, an efficient, OS-level rollbackable sandbox tailored for stateful AI agents. DeltaBox achieves millisecond-level checkpoint/rollback through a new OS abstraction, DeltaState, which treats the filesystem and process memory as a transactional, change-based state pair. To support DeltaState, we introduce two co-designed OS mechanisms in DeltaBox:

- **DeltaFS** (filesystem state management, §4.1): DeltaFS enables change-based filesystem C/R. Inspired by OverlayFS, DeltaFS organizes file states into different layers, in which higher layers will overwrite lower layers. It introduces a new feature over OverlayFS, runtime hot layer switching, dynamically freezing the current writable layer to preserve historical states and inserting a new writable layer without unmounting. A *lazy switch* mechanism based on per-inode generation counters transparently redirects active file descriptors across checkpoint boundaries. File updates are reduced to file-level CoW, and restore becomes a simple layer switch.
- **DeltaCR** (process state management, §4.2): DeltaCR enables change-based memory C/R coupled to each DeltaFS transition. At every checkpoint it performs *both* an incremental CRIU dump (for durability) and a template-creating `fork()` (for low-millisecond restore), with both costs hidden inside the LLM I/O window. A bounded template pool with LRU eviction keeps memory usage capped; evicted templates fall back transparently to the CRIU slow path, affecting only restore latency, never correctness. A background async-warm thread runs concurrently with the resumed agent, absorbing post-restore CoW faults on the agent’s writable memory regions (e.g., Python heap) off the critical path.

We implement DeltaFS as a standalone Linux filesystem and DeltaCR as a extension to CRIU, and incorporate the two key components in DeltaBox, an agent sandbox based on FireCracker microVM. End-to-end evaluations show that a coupled checkpoint completes in approximately 15 ms (mean), while repeated restores via template fork complete in ≤ 6 ms (P95 across SWE-bench workloads). On SWE-bench MCTS workloads, DeltaBox reduces the per-iteration state-management blocking time by $16\times$ (from 43 s down to 2.6 s), enabling agents to explore more search nodes. Finally, DeltaBox demonstrates robust correctness, passing all 44 functional test assertions across 15 test scenarios.

This paper makes the following contributions:

- We articulate the AI-agent sandbox C/R bottleneck, identifying that full state duplication is prohibitively slow for tree search and RL workloads. We propose the key insight of change-based DeltaState management.
- We design DeltaFS, a runtime-reconfigurable overlayfs extension enabling unmount-free layer switching and lazy file descriptor redirection.

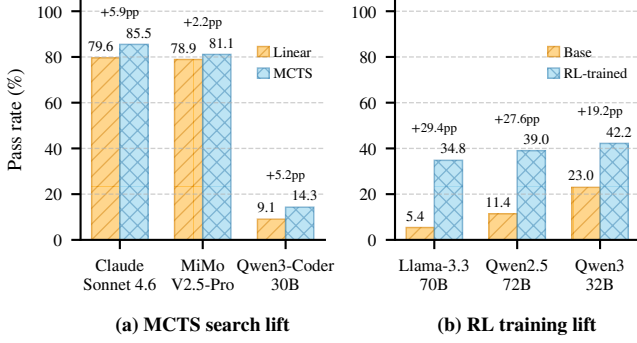


Figure 1. Pass rate on SWE-bench Verified. **(a)** Linear ReAct vs. MCTS across three coding models. **(b)** Base vs. RL-trained across three open-weight model families.

- We design DeltaCR, a process C/R system that couples incremental dumps with warm-template fork() restores and asynchronous hot-page warm-up (async-warm).
- We extensively evaluate DeltaBox on SWE-bench and on RL fan-out micro-benchmarks, demonstrating an order-of-magnitude reduction in C/R latency.

2 Background and Motivation

2.1 AI Agent Search Strategies

Modern LLM-based agents employ tree-structured search strategies to solve complex tasks. In the SWE-bench benchmark [1], agents diagnose bugs in real-world open-source projects (e.g., Django, Pandas, SymPy) and produce correct patches. Other benchmarks such as OSWorld [30], AgentBench [31], and WebArena [2] evaluate agents on desktop automation, database operations, and web navigation tasks requiring physical environment interactions.

MCTS and LATS. Modern agents have moved beyond single linear generation (Fig. 1), widely adopting systematic search strategies to explore complex task state spaces. MCTS is the classical tree-search paradigm that makes decisions through selection, expansion, evaluation, and backpropagation. Language Agent Tree Search (LATS) [6] adapts MCTS to the LLM agent domain: UCT-guided selection identifies the most promising node for expansion, where the agent generates multiple candidate actions and executes each in a sandbox. Applying this paradigm to stateful OS-level environments, rather than the text-based tasks LATS was originally evaluated on, where rollback is free, requires *fork* and *rollback* as genuine OS-level C/R operations that must complete in milliseconds to avoid blocking the LLM’s inference rhythm.

Best-of-N (BoN) and execution-guided sampling. BoN launches N independent solution trajectories from the same initial state and selects the best via an evaluation model. Recent work optimizes allocation of parallel inference samples [11], and test-time scaling more broadly drives large sample budgets [5]; agent leaderboards increasingly pair this

with per-trajectory sandboxes that execute code and observe test results. This creates a *horizontal* scaling demand: cloning the initial sandbox state into N parallel instances, with total overhead $t_{\text{checkpoint}} + N \times t_{\text{clone}}$ where t_{clone} must approach zero.

However, horizontal BoN does not eliminate the need for fine-grained C/R. It merely sidesteps it at a cost. Within each trajectory, agents perform iterative debug-test loops that mutate the sandbox state at every step. When a debugging attempt fails, the ideal response is to revert to the pre-attempt state, an intermediate checkpoint unique to this trajectory, not the initial BoN clone.

Yet no mainstream agent system provides this capability today. SWE-agent [32] uses Git stash for file-level versioning but discards process state. OpenHands [33], Aider [34], and other leaderboard systems execute trajectories linearly, with no rollback beyond restarting from scratch. When an attempt fails mid-trajectory, the only recourse is to either continue on a corrupted state or abandon the trajectory entirely and re-sample. Replaying from the initial state is expensive. Latency scales linearly with trajectory depth, and non-determinism in LLM responses and tool outputs means the replay may diverge from the original path. We observe that the absence of fast intermediate-state C/R is not a deliberate design choice but a *capability gap*: systems avoid deep search precisely because the infrastructure does not yet exist.

Test-time compute and iterative refinement. The recent shift toward test-time compute scaling [5] intensifies this gap. Reasoning models (o1 [9], DeepSeek-R1 [10]) invest more compute at inference time through extended chains of thought. Their tool-use variants execute code at each reasoning step, creating tight *iterative debug-test loops* of 5–20 iterations per trajectory. Each iteration mutates the sandbox (e.g., edit files, install packages, run processes), and backtracking from a failed attempt requires restoring the pre-attempt state, which is a capability no current sandbox platform efficiently provides. As a result, existing systems truncate search depth: they either run linear trajectories without rollback, or rely on coarse-grained BoN with independent restarts, leaving significant compute budget on the table.

DeltaBox (a new agent sandbox system proposed in this paper) is designed to close this gap, enabling a workflow that combines *outer BoN* (horizontal cloning of the initial sandbox) with *inner iterative refinement* (vertical checkpoint/rollback within each trajectory), requiring fast C/R at two distinct granularities that current infrastructure cannot deliver.

2.2 Agent Sandbox

Agent sandbox, as usual system-level sandboxes like Docker container, FireCracker [17, 27], gVisor [16], and many others [15, 35–37], can provide an isolated and (usually) self-contained environment to: (1) run a standalone agent; or (2) be used by an agent to execute tools or code which may be

Table 1. Comparison of sandbox state management approaches for AI agents. DeltaBox provides coupled checkpoint/restore of both filesystem and process state by only duplicating the changes between consecutive checkpoints for efficiency.

Approach	Ckpt Latency	Restore Latency	Write Amplif.	FS State	Process State	Arbitrary Rollback	Agent Transparent
Git stash/branch	100ms–1s	100ms–1s	Low	✓	✗	✓	✗
shutil.copytree	100ms–10s	100ms–10s	$O(\text{dir})$	✓	✗	✓	✗
Docker commit + restart	1–10s	1–10s	$O(\text{layer})$	✓	✗	✓	✗
Btrfs/LVM snapshot	10–100ms	10–100ms	Low	✓	✗	✓	✗
Firecracker VM snapshot [15]	200ms–2s	120–700ms	$O(\text{VM})$	✓	✓	✓	✓
LangGraph Checkpointer	<1ms	<1ms	—	✗	Logical [†]	✓	✗
DSec [28] [‡]	— [‡]	WAL replay	$O(\log)$	✗	✗	✗	✓
CubeSandbox [29] [§]	148–226 ms [§]	<60ms	$O(\text{VM})$	✗	✗	✗	✓
DeltaBox (ours)	14.57 ms	5.14 ms[¶]	$O(4\text{KB})$	✓	✓	✓	✓

[†]Saves Python graph state only; cannot undo OS-level side effects. [‡]DSec [28] recovers via WAL replay of cached outputs; latency proportional to replay depth. [§]Ckpt measured via cloud-hypervisor (CubeSandbox’s VMM): pause+snapshot+resume, 512 MB VM, 256 MB dirty; range 148–226 ms, median 154 ms (5 runs). ^{||}Published <60 ms is cold-start latency, not event-level snapshot rollback; the latter is on CubeSandbox’s roadmap. §6.2.1 approximates the planned mechanism via the underlying VMM + dm-snapshot. [¶]Fast path: 5.14 ms (FS switch 1.66 ms || template fork 3.75 ms). Slow path (CRIU lazy-pages): 8.04 ms (FS switch 1.66 ms || CRIU restore 7.25 ms). Both means across SWE-bench workloads; see Table. 4 for component breakdowns.

dangerous to be executed in native host. In this paper, we will not explicitly distinguish between the two usages unless necessary.

As a result, agent sandbox state is *two-dimensional*: every search tree node corresponds to a *joint* (filesystem, memory) state that must be saved and restored atomically.

Filesystem state. The agent’s working directory (e.g., a cloned Git repository with hundreds of thousands of files) constitutes the filesystem state. Each agent action (editing source code, running `pip install`, modifying configuration files) mutates this state.

Process state. The agent process maintains in-memory state including the LLM conversation history, parsed tool outputs, intermediate variables, and open file descriptors. Losing this state upon rollback forces the agent to replay its entire action history from the initial state, adding latency proportional to the search depth and incurring redundant LLM API calls.

Why coupling matters. If only filesystem state is rolled back (without process memory), the agent process continues with stale in-memory context; it “remembers” modifications that no longer exist on disk. Conversely, if only process state is restored (without filesystem rollback), the agent operates on files from a different search branch. Either mismatch breaks the determinism required for correct tree search. As our experiments confirm (§6), filesystem-only rollback without CRIU memory restore produces observable semantic inconsistency.

2.3 Limitations of Existing Approaches

Table.1 summarizes the characteristics of existing sandbox state management approaches along both dimensions.

Git-based approaches. SWE-Agent [32] uses Git stash and branch operations for filesystem versioning. Git provides

semantic versioning for text files but cannot capture binary files, installed packages, or process state.

File-level copying and Docker commit. Naive file copying (e.g., `shutil.copytree`) and Docker commit both provide filesystem snapshots but neither preserves process memory. The agent must be restarted, replaying its full history.

Firecracker VM snapshots. Firecracker [27] is a light-weight VMM for microVMs. Its snapshot/restore captures the complete guest state at VM granularity, tracking guest physical memory (e.g., dirty pages) rather than a single application process. The snapshot granularity is the entire VM: the monitor observes all guest physical pages, including kernel page table updates, slab allocator activity, and background daemon memory, much of it irrelevant to the agent’s actual state. We include Firecracker’s snapshot API as a baseline in §6; the quantitative comparison against process-level DeltaCR appears as the Firecracker Diff column in Table. 2.

LangGraph logical checkpoints. LangGraph [38] provides application-level checkpointing by serializing the Python graph state (message lists, node outputs, accumulated variables). This is extremely fast (<1 ms) but *cannot undo physical side effects*: a `sed` command that modified a file is not reversed by restoring the graph checkpoint. DeltaBox and LangGraph are complementary, not competing (§4.4).

2.4 Design Requirements

Based on the above analysis, we identify four key requirements for an efficient agent sandbox:

R1 Sub-100 ms coupled checkpoint/restore. Both filesystem and process state must be saved/restored jointly within this budget.

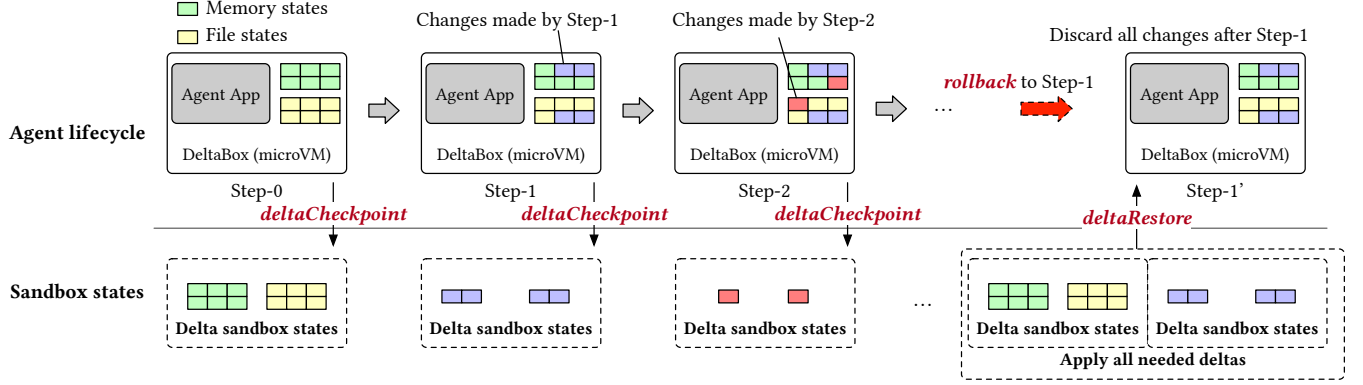


Figure 2. Design Overview. DeltaBox utilizes diff-based checkpoint/restore (i.e., `deltaCheckpoint` and `deltaRestore`) to enable millisecond-level sandboxbacks. An agent application can fully run inside a sandbox, or utilize a sandbox to execute a set of tools and use the C/R capabilities of DeltaBox to ensure prompt rollback when necessary.

R2 Write amplification proportional to actual changes.

Storage overhead must scale with the agent’s actual modifications, not working directory size.

R3 $O(1)$ arbitrary rollback. Support rollback to any historical checkpoint in (near) constant time.

R4 Agent transparency. The coupled checkpoint/restore should be mostly invisible to the agent, i.e., no code changes, no forced restarts, and no context loss.

3 System Overview

This paper presents DeltaBox, a new agent sandbox that satisfies the four requirements.

3.1 Insights

We observe that subsequent checkpoints in AI agent workloads are highly similar, with only minor incremental changes between steps (e.g., a few new files or modified memory pages). Therefore, to achieve efficient checkpoint and rollback, instead of duplicating the entire state, a sandbox should only duplicate the changes between consecutive checkpoints.

Fig. 2 sketches DeltaBox’s end-to-end workflow following the key insight. Concurrently with each LLM round-trip in the agent’s lifecycle, the StateManager issues a `deltaCheckpoint` that captures and persists only the *delta* of the coupled (memory, filesystem) state produced by that step. A rollback is realized by `deltaRestore`: the system first switches the OverlayFS layer stack to the target state in one shot, and then reconstructs the target process memory either by forking a warm template (on hit) or by replaying the incremental CRIU image chain, thereby restoring that step’s exact (memory, filesystem) view. The figure also illustrates the two usage modes DeltaBox supports, both sharing the same primitives: the agent can run *entirely inside* a sandbox so that every step is automatically C/R-protected, or a host-side agent can drive the sandbox *as a tool executor*,

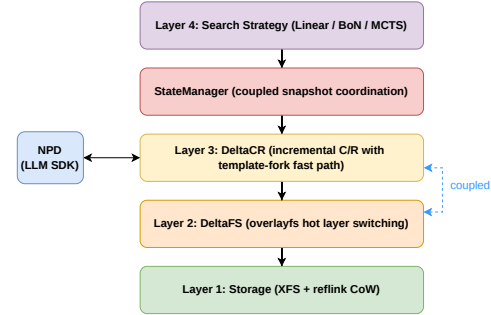


Figure 3. The DeltaBox architecture. The StateManager coordinates DeltaFS (Layer 2, filesystem state) and DeltaCR (Layer 3, process state) to maintain consistent (filesystem, memory) state pairs at each search tree node. Base storage (Layer 1) provides the real storage functionalities. It would be better to adopt XFS (with reflink) to achieve block-level CoW and eliminate write amplification.

invoking `deltaRestore` to undo any unwanted side effect from an individual tool call (§4.4).

3.2 System Architecture

DeltaBox adopts a four-layer architecture to manage checkpoints and realize efficient checkpoint and rollback, as shown in Fig.3. DeltaBox enforces a core property: every checkpoint is a consistent (filesystem, memory) state pair, and DeltaBox only tracks the *delta* of the state between consecutive checkpoints. A StateManager coordinates both dimensions to ensure atomicity.

Layer 1: Base storage. All DeltaFS layers reside on a real filesystem, e.g., XFS with reflink in DeltaBox’s prototype. When DeltaFS performs a copy-up, XFS creates shared block references via reflink rather than duplicating data, deferring physical block allocation to the point of actual write (4 KB granularity).

Layer 2: DeltaFS (filesystem state management). DeltaFS is a new filesystem layer extended based on Linux overlays.

A key capability of DeltaFS is the *runtime reconfiguration* of the overlay layer stack, i.e., DeltaBox can insert or remove overlay layers at runtime, which is not allowed in Linux overlaysfs (§4.1). A lazy switch mechanism handles open files across checkpoint boundaries (Fig. 4.1).

Layer 3: DeltaCR (process state management). The symmetric counterpart to DeltaFS, DeltaCR provides process-level checkpoint/restore atop CRIU (§4.2). On checkpoint, DeltaCR performs *both* an asynchronous CRIU incremental dump to tmpfs and a template-creating fork(), so every node acquires a durable image and a frozen template at near-zero marginal cost (the combined latency is hidden inside the LLM I/O window). On restore, DeltaCR fork()s the template (low-millisecond); if the template has been evicted from the bounded pool, it falls back to CRIU lazy-pages restore (~8 ms). A background *async-warm* thread runs concurrently with the resumed agent, walking the child’s anonymous writable VMAs to absorb CoW faults off the agent’s critical path. DeltaBox further employs adaptive optimizations (semantic-aware skip, chain management, garbage collection) reduce overhead in long-running searches.

StateManager (coordination). The StateManager is realized as a two-tier architecture: a Host-side *Sandbox Controller* for global coordination and a Guest-side *Guest State Daemon* (GSD) for local execution (§4.4). The Sandbox Controller maintains a global snapshot index tree isomorphic to the search tree. Each node stores: (a) a snapshot ID, (b) the CRIU dump directory path, (c) the overlaysfs layer stack configuration (upper + lower paths), and (d) metadata (parent ID, creation timestamp, branch status). The GSD executes the coupled checkpoint/restore sequences described below entirely within the VM, without host involvement on the critical path.

3.3 Coupled deltaCheckpoint Flow

We first present the overall checkpoint flow in DeltaBox.

1. The search strategy (of an agent) issues a checkpoint request to the StateManager; the Sandbox Controller dispatches it to the target VM’s GSD.
2. **DeltaCR:** The GSD submits an incremental CRIU dump to a single-worker background thread pool. CRIU briefly SIGSTOPs the agent on entry, writes the dump to tmpfs, and SIGCONTs the agent on completion. The GSD does not block on the dump and proceeds immediately to the next step.
3. **DeltaFS:** The GSD synchronously issues a DeltaFS ioctl on the calling thread. The kernel atomically: (a) saves the current upper layer of the FS as a new read-only lower, (b) installs a fresh upper, (c) bumps checkpoint_gen (details in §4.1), (d) invalidates potential caches (e.g., directory caches).
4. **DeltaCR (template creation).** The GSD waits for the dump to complete, then writes a fork request to the

agent’s control FIFO. The agent fork()s at a quiescent point: the parent self-suspends as a *template*; the child becomes the new active agent. If the bounded template pool (N_{tpl} entries) is full, the least-recently-used template is evicted (SIGKILL’d); the evicted node retains its checkpoint image for slow-path restore.

5. **StateManager** registers the checkpoint: {ID, CRIU dump path, layer config, template PID if any}.

State consistency. The (filesystem, memory) pair captured at checkpoint k is consistent because CRIU’s internal SIGSTOP barrier in step 2 freezes file I/O and memory mutations at a single instant; the dump that follows reflects exactly that instant. The ioctl in step 3 only reconfigures the overlay layer-stack pointer (not the file contents), and the renamed prior upper preserves the data on disk as a new read-only lower, so the filesystem image at checkpoint k matches the memory image timestamp.

Inference-masked checkpointing. Checkpointing an AI agent typically occurs during key state transitions [39], such as reaching a milestone, invoking an LLM, or executing tools. This presents a critical optimization opportunity: by checkpointing while the agent waits for an LLM response, the system can effectively hide the C/R latency. We call this *inference-masked checkpointing*. The primary challenge is preserving the active network connection to the remote LLM during the checkpoint. DeltaBox resolves this by isolating the HTTPS connections within a separate Network Proxy Daemon (§4.2). LLM requests progress uninterrupted while CRIU dumps the agent process. Because the agent’s SIGSTOP window is extremely brief (~15 ms), it easily fits within the LLM’s response time, masking the checkpoint overhead.

3.4 Coupled deltaRestore Flow

The restore flow in DeltaBox is as follows:

1. The search strategy (of an agent) issues a restore request to the StateManager targeting a prior checkpoint; the Sandbox Controller dispatches to the GSD.
2. The GSD kills the current agent process (SIGKILL).
3. **DeltaFS:** The ioctl switches the overlay layer stack to the target checkpoint’s configuration, restoring the exact filesystem state.
4. **DeltaCR (fast path):** If the target checkpoint’s frozen template is alive, fork() it. A background *async-warm* thread runs concurrently with the resumed agent, touching hot-zone pages to absorb their CoW faults off the critical path.
DeltaCR (slow path): If the template has been GC’d, CRIU lazy-pages restore creates the process from dump images, with hot-first eager prefetch.
5. The agent resumes execution from the exact instruction after the original checkpoint, unaware of the rollback. Both its memory and its filesystem view are consistent.

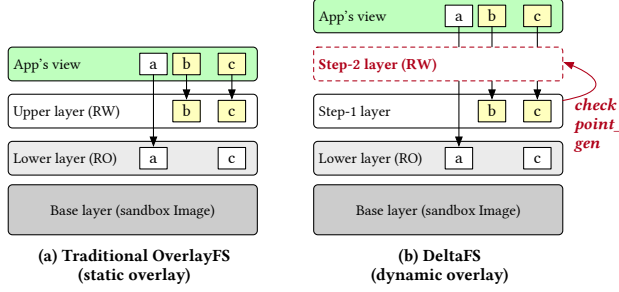


Figure 4. DeltaFS architecture. (a) Traditional overlayfs will prepare an upper layer file system which is writable for apps, and maintain a lower layer file system which is read-only and basically includes everything in the sandbox image. (b) DeltaFS extends the idea to support dynamic overlay, i.e., when an agent finishes a step of task and needs to make a checkpoint, instead of duplicating all files, DeltaFS inserts a new writable layer above existing one and frozen the existing wrtiable layer into a read-only state, and all following updates are copy-on-write to the new upper layer. As a result, the checkpoint operation is simply a layer insert operation while a rollback is a set of layer remove operations.

4 Detailed Design

This section presents the detailed design of DeltaBox.

4.1 DeltaFS: Dynamic Overlay Filesystem for Agents

Standard Linux overlayfs freezes its layer stack at mount time; modifying it requires `umount/mount`, which demands no process hold an open file under the mount, flushes the dentry and inode caches, and costs tens of milliseconds per cycle, unacceptable when MCTS issues hundreds to thousands of checkpoints.

Runtime layer switching. DeltaBox extends the overlayfs kernel module with a custom `ioctl` that reconfigures the layer stack *without unmounting*. The `ioctl` accepts a configuration string in mount-option format (`lowerdir=...`, `upperdir=...`, `workdir=...`). It executes in four steps. First, it parses the option string and resolves the paths of the new layers. Second, it allocates a private mount clone for each path to build a new layer array. Third, it performs an atomic swap: it publishes the new array via release-consistent stores, increments the per-filesystem `checkpoint_gen` counter (red annotation in Fig. 4b), and invalidates stale dentry and inode caches. Finally, it defers cleanup of the old layer array until two checkpoints later, relying on mount reference counts to protect any in-flight readers accessing open backing files.

The user-space controller renames the current upper directory to a new read-only lower *before* issuing the `ioctl`, so only metadata operations appear on the critical path; the `ioctl` also splices the demoted upper as the topmost lower, preserving access to files copied-up before the checkpoint.

Lazy switch for open files. Files (or mmap'd regions) opened before a checkpoint retain stale inode metadata after hot-switch, since their cached upper-dentry pointers reference the demoted upper. A per-filesystem generation counter (`checkpoint_gen`) resolves this. Each inode caches the generation at which it was last resolved. On the write path, the kernel compares this cached generation against the current filesystem generation. If the generations match (the fast path), the operation proceeds directly. Conversely, if a mismatch occurs (the slow path), the kernel re-resolves the inode against the new layer stack, allocating a fresh upper file via copy-up if necessary. The inode's generation and its backing dentry are then updated atomically under a per-inode mutex.

Concurrent copy-up of the same dentry races: the second thread sees `EEXIST`. We discriminate by comparing the existing upper file's backing mount against the current overlay upper. If they match, the upper was created by a same-generation concurrent copy-up and is reused; otherwise it is a stale pre-checkpoint upper, and we clear it so copy-up restarts on the new upper layer.

XFS reflink. While DeltaFS can utilize most file systems as a backing store, DeltaBox specifically uses XFS with reflink enabled to bound per-checkpoint write amplification. Overlayfs copy-up utilizes `vfs_clone_file_range`, which on reflink-enabled XFS clones extents by reference. An upper file therefore shares all physical blocks with its source until explicitly overwritten. When hot-switch demotes the upper layer via `rename(2)`, it preserves the file's extent map, meaning the demoted file enters the new lower chain with its reflink edges intact. Because reflink composes transitively, an extent that remains unmodified across N checkpoints occupies only a single physical block shared by all N generations. This extent-map preservation is crucial: it caps the write amplification plateau to $\sim 170\times$ for large files regardless of file size (§6.3.2). Without this property, write amplification would scale linearly with the checkpoint count.

4.2 DeltaCR: Diff-based Checkpoint/Restore

DeltaCR manages the process (memory) dimension of each checkpoint. It builds on CRIU [26], which provides incremental dumps via soft-dirty page tracking and lazy-pages restores via `userfaultfd`. DeltaCR introduces three key mechanisms: (i) a *warm-template* fast path that preserves checkpointed processes as frozen templates, enabling low-millisecond restores via OS-level `fork()`; (ii) a GSD-side async-warm thread that runs concurrently with the resumed agent to absorb CoW faults on its anonymous writable pages (§4.2.2); and (iii) a Network Proxy Daemon that decouples LLM I/O from the agent's address space, ensuring templates remain safely forkable (§4.2.3). Fig. 5 summarizes these components and the two restore paths.

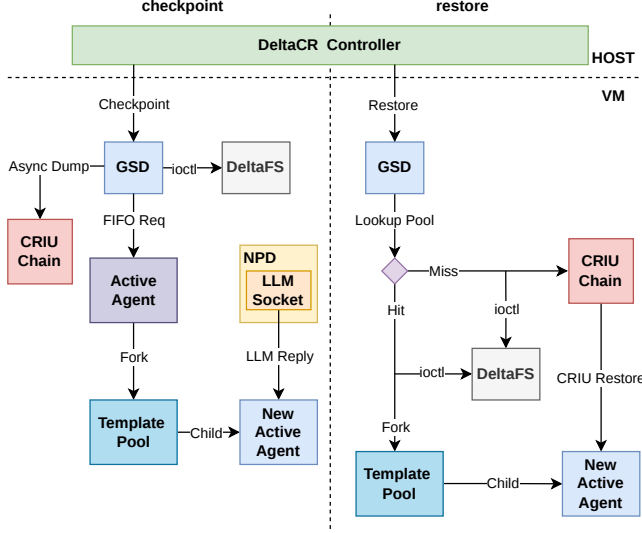


Figure 5. DeltaCR architecture. Checkpointing creates both a CRIU image chain and a frozen template. Restores use the template fast path on hit, or the CRIU chain on miss; NPD keeps external I/O off the agent path.

Dual-path checkpoint. At every checkpoint, DeltaCR simultaneously performs an asynchronous CRIU incremental dump and a template-creating `fork()`. The CRIU dump provides a durable image for crash recovery and slow-path restores, while the frozen template enables low-millisecond fast-path restores via `fork()`. The combined execution time (~ 10 ms for the dump and 3–4 ms for the fork) is entirely hidden within the LLM inference window (inference-masked checkpointing, §3.3), so every search tree node acquires both a durable image and a frozen template at near-zero marginal critical-path cost.

Bounded template pool. To manage memory overhead, frozen templates are stored in a bounded pool sized to a configurable limit, N_{tpl} . When a new template exceeds this bound, DeltaBox evicts the LRU entry. Because the evicted node’s CRIU dump image remains safely on disk, any future restore targeting that node transparently falls back to the CRIU lazy-pages slow path (~ 8 ms, Table. 4).

4.2.1 Template Pool and Fork-Based Restore The fast-path restore mechanism relies on a *template pool*, which maintains a registry of frozen (SIGSTOP’d) processes, each keyed to a specific snapshot ID.

Checkpoint flow. During checkpoint k , DeltaCR executes a CRIU incremental dump to `tmpfs` as a durable fallback for crash recovery and cold-path restores. CRIU’s -leave-running mode internally SIGSTOPS the agent only for the dump and resumes it on completion, so the agent process safely continues. Concurrently, the GSD writes a fork request to the agent’s control FIFO; the agent processes it at its next quiescent point—no syscalls in flight, no files half-written—and inline-forks. The parent

SIGSTOPS and registers as *template_k* in the pool; the child returns to the main loop as the newly active agent. Page tables are duplicated but no physical memory is copied (copy-on-write); the template and child initially share all pages.

Fast-path restore (template hit). When the search strategy requests a restore to checkpoint k and *template_k* is alive, the GSD kills the current active agent. It then issues the DeltaFS `ioctl` to revert the overlays layer stack to checkpoint k ’s exact configuration. Following this, DeltaCR executes a `fork()` on *template_k* to spawn a new child process, P'' . P'' is resumed via SIGCONT and continues execution from the exact instruction immediately following the original checkpoint, with both memory and filesystem states consistent. Concurrently with P'' ’s execution, a background async-warm thread (§4.2.2) walks the child’s hot-zone pages to absorb their CoW faults off the critical path. This `fork()`-based restore completes in low-millisecond time regardless of the process’s *Resident Set Size (RSS)*, as the kernel only duplicates the page tables themselves, not the memory contents. Unlike CRIU dumps, which capture all threads, `fork()` only copies the calling thread. By initiating the fork from the agent’s own main loop at a quiescent point, DeltaBox avoids the multi-thread fork hazard where a thread frozen mid-mutex by an external SIGSTOP could deadlock the child.

Slow-path restore (template miss). If *template_k* has been garbage-collected, DeltaCR falls back to the CRIU lazy-pages restore path. CRIU reconstructs the process skeleton, registers all anonymous VMAs with `userfaultfd`, and resumes the process, faulting in pages on demand from the `tmpfs` dump images. To minimize critical-path latency, the background async-warm thread (§4.2.2) prefetches hot-zone pages off the critical path. Once the restored process stabilizes, it is frozen and injected back into the template pool to serve future fast-path restores.

Template lifecycle. The template pool maps snapshot IDs to frozen-process PIDs, with lifetimes dictated by reachability in the search tree. When the search strategy prunes a subtree, a garbage collector automatically removes the corresponding entries and SIGKILLS their templates.

4.2.2 Async-warm Following a `fork()`, the template and the active child share all pages via copy-on-write (CoW). The child’s first write to any shared page therefore incurs a synchronous kernel-side fault to execute the physical copy, taking several microseconds. In a typical Python agent (~ 800 MB RSS), a significant fraction of these pages may be written within the agent’s first post-restore turn; left unmanaged, the resulting CoW faults scatter latency across the agent’s critical path.

To absorb this overhead, DeltaCR introduces a *GSD-side async-warm thread*. The thread is spawned within the GSD rather than the agent, keeping the agent strictly single-threaded for subsequent checkpoints, and it runs on

a separate CPU off the agent’s critical path. Immediately after the `fork()`, the GSD launches the `async-warm` thread which iterates through the child’s anonymous writable VMAs in `/proc/[pid]/mem` concurrently with the resumed child. For every page, the thread reads and rewrites a single byte. Reading the byte first ensures the rewrite preserves contents bit-identically, while the write forces the kernel to execute the CoW physical copy and privatise the page in P'' ’s address space.

The `async-warm` runs purely in the background: pages it has already privatised cause no fault when the agent later writes them; pages it has not yet reached follow the normal CoW path, so the worst case degenerates to plain CoW (no penalty over a no-warm baseline). For agents whose post-restore work has any non-trivial idle window, the `async-warm` completes before the bulk of agent heap writes, absorbing most CoW faults from the critical path; we quantify the residual fault count in §6.3.1.

4.2.3 Reusable I/O during Checkpoint with NPD A critical challenge for `fork()`-based restores is ensuring that the frozen template’s address space contains no long-lived external state, such as active network threads. However, standard LLM SDKs in Python (e.g., `openai` or `anthropic`) violate this requirement by spawning background HTTP/2 connection pools and `ThreadPoolExecutor` workers. Forking a template containing these threads leads to fatal issues: keep-alive TCP sockets survive as half-open file descriptors, connection-pool states diverge, and threads frozen mid-handshake permanently deadlock the parent template.

To resolve this, DeltaBox decouples LLM API I/O from the agent’s address space by routing all traffic through a dedicated *Network Proxy Daemon* (NPD) residing in the same sandbox. The agent process never links the upstream SDKs directly; instead, it communicates with the NPD via atomic FIFO tokens and a shared memory-mapped directory. Because the NPD is an entirely separate process, it is excluded from both the CRIU dump and the template `fork()`. The templated agent thus remains bounded to its own short-lived thread set, free of persistent socket states. During a checkpoint, only the agent is frozen. The NPD continues to receive and buffer API responses asynchronously, ensuring that checkpoint latency is hidden beneath the in-flight LLM inference time. Upon a `fork()`-based restore, the new child inherits the FIFO descriptors directly from the template, allowing transparent communication resumption without requiring any reconnection protocol. DeltaBox does not currently support network I/O rollback, which may incur external side effects.

4.3 StateManager Coupling Protocol

The StateManager coordinates the system to enforce a strict invariant: *every saved state is a consistent (filesystem, memory) pair*.

Consistency protocol and failure handling. Consistency relies on tightly synchronizing the two primitives during checkpoint and restore. During a checkpoint, the asynchronous CRIU dump and the synchronous DeltaFS `ioctl` both observe the agent at the exact same SIGSTOP-quiesced instant, ensuring the persisted memory image matches the frozen upper layer. During a restore, the `ioctl` switches the filesystem layer stack *before* the new agent process resumes, guaranteeing the agent never executes against mismatched files. If the CRIU dump fails (e.g., due to an incompatible resource), the StateManager gracefully aborts by rolling back the filesystem `ioctl` and reporting the error to the search strategy, preventing any inconsistent half-states.

Value-time test isolation. Search frameworks often evaluate states by running tests (e.g., SWE-Search [40]), which generate unwanted side effects like `__pycache__` or temporary files. In stateless, sandboxed environments, these side effects are naturally isolated by discarding the sandbox. Because DeltaBox operates a stateful, in-process environment, the StateManager explicitly provides isolation using the C/R primitive itself. Before executing a test, the StateManager takes a pre-test checkpoint. Once the agent reads the test results, the StateManager unconditionally restores to the pre-test checkpoint. Because the agent is quiesced while awaiting the test observation, resuming it from the pre-test state and injecting the result mimics a side-effect-free execution.

4.4 Compatibility and Deployment Model

Compatibility with agent frameworks. DeltaBox is designed to complement application-level agent frameworks like LangGraph [38] and LangChain [41]. While tools like LangGraph’s Checkpointer serialize logical Python states (e.g., message lists, node outputs), they fundamentally cannot undo physical OS-level side effects like modified files or installed packages. DeltaBox bridges this gap by providing the necessary physical state management layer. We expose DeltaBox to these frameworks via two integration modes: a transparent adapter that wraps the framework’s native checkpoint saver to trigger OS-level C/R automatically, and an explicit tool node that grants developers explicit control over when to capture OS-level snapshots.

Deployment model. To balance global search scalability with low-latency local operations, DeltaBox adopts a Host-Guest collaborative architecture. The StateManager is split accordingly: a Sandbox Controller on the Host manages global coordination and the snapshot index tree, while a GSD inside each guest VM handles the local C/R execution. The system relies on Firecracker [27] microVMs for strict hardware-level isolation, with each VM running a custom Linux 6.8 kernel that natively integrates the DeltaFS module and DeltaCR primitives.

5 Implementation

To realize DeltaBox, we implement two key modules, DeltaFS (filesystem state) and DeltaCR (process state), coordinated by a StateManager, and utilize FireCracker [27] microVMs as the base sandbox system. The key optimizations in DeltaBox can also be used by container-based sandbox systems (e.g., deploying DeltaFS for docker container) or other VM-based systems. DeltaFS is based on Linux 6.8 overlayfs module; while DeltaCR orchestrates CRIU, the template pool, and the async-warm thread.

5.1 DeltaFS: Overlayfs Kernel Module

The checkpoint `ioctl` implements the four-phase flow described in § 4.1. Each new layer receives a private mount clone to avoid aliasing with external mount references. The layer-array pointer is exchanged atomically under a spin-lock; the old array is freed after an RCU grace period. After the swap, all cached directory entries are marked for revalidation on next access, ensuring stale upper references are never served from cache.

The lazy file switch intercepts the write path. On a generation mismatch, it re-resolves the inode against the new layer stack under a per-inode mutex. The concurrent copy-up race (two threads creating the same upper file) is handled by comparing mount-namespace identity: if the upper already exists from a concurrent copy-up in the same namespace, it is reused.

5.2 DeltaCR Userspace Daemon

Template pool. A pool maps snapshot IDs to frozen-process PIDs. On checkpoint, after the CRIU dump completes, the GSD writes a fork request to a control FIFO. The agent picks up the request at the next quiescent point of its main loop and calls `fork()` inline; the parent self-suspends with `SIGSTOP` and is registered as a template, while the child returns to the main loop as the new active agent. This keeps the mechanism multi-thread-safe and transparent to agent application code.

Fast-path restore. On `restore(target_id)`, if a template exists for the target, the GSD signals the template, which forks a new child. The GSD then launches the async-warm thread targeting the child’s PID.

Async-warm thread. The async-warm thread walks the new child’s anonymous writable VMAs at restore time and reads-then-rewrites one byte per page via `/proc/[pid]/mem`, triggering CoW page faults in bulk while preserving page contents bit-identical. The thread runs in the GSD process on a separate CPU and proceeds concurrently with the resumed child.

Slow-path restore (CRIU fallback). When a template is unavailable, DeltaCR invokes CRIU’s lazy-pages restore mode, which creates the process skeleton with `userfaultfd`-backed pages. The async-warm thread then prefetches the agent’s writable pages from the tmpfs dump images concurrently

with the resumed agent. After restore stabilizes, the process is frozen and added to the template pool. DeltaCR’s contribution layers over existing CRIU primitives (incremental dump, lazy-pages, `userfaultfd`, soft-dirty): the dual-path checkpoint and the LRU template pool coupled to DeltaFS layer transitions, with CRIU lazy-pages retained as a transparent fallback.

Network Proxy Daemon (NPD). The NPD runs the LLM SDK clients on the agent’s behalf, exposing a control surface to the agent through a pair of FIFOs (request and notify) and a shared request/response file directory. Agent processes never import those SDKs directly, so the agent’s address space never contains SDK-side worker threads or connection pools, keeping the template’s residual thread set bounded and forkable. The NPD also buffers in-flight responses across checkpoint freezes and re-pairs the agent-side socket after a fork-based restore.

Semantic classifier. A lightweight rule-based classifier inspects the agent’s next action. Commands matching read-only patterns (`grep`, `cat`, `find`, `ls`, `git diff`, `python -m pytest --collect-only`) trigger the lightweight skip path.

Garbage collector. A background thread periodically scans the snapshot index tree for pruned subtrees. For each pruned node it `SIGKILLs` the template process (if any), deletes the CRIU image directory, and removes the corresponding overlayfs layer directory. Ancestors of any live descendant on the incremental dump parent chain are preserved lazily: a pruned node is held back if any kept node still references it, and is only freed once the chain no longer needs it.

Snapshot storage layout. Each microVM stores CRIU dumps in tmpfs and overlayfs upper directories on an XFS-formatted rootfs block device, where `relink CoW` operates at block granularity (Fig. 4.1).

5.2.1 MCTS-aware garbage collection Snapshot storage grows with the number of live tree nodes, so some form of garbage collection is required for any long-running search. Unlike linear, best-of-K, beam, or DFS search (where selection is monotonic and a node, once dropped, is never revisited), MCTS may re-select any previously-expanded non-terminal node whose Q value subsequently rises via descendant backpropagation. The bounded template pool (§ 4.2) handles this gracefully: evicting a template from the pool does not delete the node’s CRIU dump image, so UCT re-selection of an evicted node simply falls back to the CRIU lazy-pages slow path (~8 ms) rather than failing. Template eviction therefore affects only restore *latency*, never *correctness*. The remaining GC concern is *snapshot storage*: CRIU dump directories and overlayfs layer directories of truly unreachable nodes should still be reclaimed. **Any recency- or visit-count-based storage GC policy is unsafe for MCTS by construction:** evicting a node’s dump images leaves its Q and visit count in the search tree, after which UCT re-selects

the same node, the restore fails with an “Unknown ID” error, and because the aborted iteration does not increment visits, UCT re-selects the same node again on every subsequent iteration until the budget expires.

We propose a *reachability-aware* keep rule that couples GC to the search algorithm’s own selection semantics. At each GC pass, the controller computes the set of MCTS-reachable state IDs—exactly the nodes UCT’s selection rule may still return, i.e., non-terminal, non-failed, non-duplicate nodes that have not exhausted their expansion budget and whose children have not already reached the reward threshold—together with all terminal nodes retained as final-patch candidates for the discriminator. The controller preserves this set plus every ancestor on the CRIU `prev-images-dir` chain and evicts everything else. The rule is safe by construction: the only nodes discarded are those the search algorithm itself has declared unreachable.

5.3 Integration with Agent Frameworks

We provide two integration adapters for LangGraph [38]:

(a) Transparent Checkpointer. A Python class implementing LangGraph’s `BaseCheckpointSaver` interface. Its `put()` method calls `StateManager.checkpoint()` to create a coupled OS-level snapshot alongside the graph-state checkpoint. Its `get()` method calls `StateManager.restore()` to roll back both physical and logical state.

(b) Tool wrapper. DeltaBox checkpoint/restore is exposed as LangGraph tool nodes. The agent or the search strategy can invoke `sandbox_checkpoint` and `sandbox_restore` as explicit graph actions, providing fine-grained control over when OS-level snapshots are created.

6 Evaluation

6.1 Experimental Setup

Hardware. All experiments run on a server with a 14-core x86 CPU and 32 GB RAM. RL training fan-out (Fig. 8b–c) additionally uses a single-node 4-GPU cluster (96 GB each); T_{train} at $B \geq 16$ runs FSDP weight-sharded across all four GPUs because single-GPU activation memory at $B=16$ seq 768 exceeds 96 GB. Each sandbox instance (DeltaBox based on Firecracker microVM) is configured with 4 vCPUs, 4 GB RAM, and a single XFS rootfs block device.

Workloads. We focus on four representative *primary workloads* drawn from SWE-bench Verified, covering the main MCTS-trajectory profiles:

- **Django (fat process):** A web-framework repair task. Moderate working directory (~358 MB) but complex process state (many opened file descriptors, loaded modules).
- **Sympy (read-heavy exploration):** A symbolic-math repair task whose MCTS trajectory is dominated by read-only inspection actions (`grep` / `cat` / `test` discovery); see §6.3.3 for per-archetype lightweight-skip measurements.

- **Astropy / Xarray (scientific stack):** Two scientific-computing repair tasks with heavy NumPy-backed import surfaces and modest working directories. They exercise the same MCTS-trajectory profile (process state dominates).

Broader sweeps additionally cover up to 10 SWE-bench Verified repositories (adding Flask, Matplotlib, scikit-learn, pytest, pylint, Sphinx for the write-amplification and production-MCTS studies); see individual figure captions for per-experiment scope.

Baselines.

- **copytree+replay:** Filesystem state via `shutil.copytree` duplication of the testbed directory; process state recovered by re-executing recorded `EditCode` commands (no LLM re-invocation; commands are persisted in the trace).
- **docker+replay:** Filesystem state via `docker commit` (new image layer per checkpoint, restored via `docker stop/rm/run` from the committed image); process state recovered by re-executing recorded `EditCode` commands inside the restored container.
- **Firecracker+dm-snapshot:** VM-level memory snapshot via Firecracker’s built-in pause+dump mechanism, coupled with a `dm-snapshot` CoW layer for filesystem state. Both Full and Diff Firecracker modes are reported in Table.2.
- **CRIU+File copy (shutil.copytree):** Filesystem state via `shutil.copytree`; process state via CRIU dump and restore on the agent process.
- **CRIU+Docker commit:** Filesystem state via `docker commit`; process state via CRIU dump and restore on the in-container agent process inside a privileged container.

6.2 End-to-End Performance

DeltaBox’s coupled checkpoint/restore primitive is designed to serve two distinct multi-iteration agent workloads: (i) inference-time MCTS search, where each tree expansion checkpoints at a parent node and restores at a new leaf; and (ii) RL training fan-out, where each training step forks N parallel rollout sandboxes from a single warm template. Both stress the per-event primitive at high frequency.

6.2.1 MCTS Search Throughput We characterize the per-iteration blocking overhead each sandboxing approach contributes to MCTS search, and corroborate Table.2’s replay-bench numbers against an end-to-end trajectory-replay experiment.

Table 2 decomposes per-event latency into checkpoint and restore phases. DeltaBox’s `std ckpt` is dominated by an asynchronous incremental CRIU dump that is masked by LLM inference (§4.2); `LW ckpt` collapses further since pure-read commands leave the upper clean. The `std path` additionally creates a warm template at checkpoint—the agent self-forks; the parent SIGSTOPS as the snapshot’s frozen template, the

Table 2. Per-event mean blocking time (ms) on SWE-bench MCTS trajectories. *ck/rs* = checkpoint/restore; column naming follows *process-recovery+FS-recovery* (see §6.1 for baseline definitions). DeltaBox columns split by classifier: *LW* = pure-read actions (entire checkpoint skipped); *std* = file-mutating actions (full incremental CRIU dump); restore splits the same way by the target ckpt’s tag.

Workload	replay+cp		replay+dk		FC-Full+dm		FC-Diff+dm		CRIU+cp		CRIU+dk		DeltaBox			
	ck	rs	ck	rs	ck	rs	ck	rs	ck	rs	ck	rs	LW ck	LW rs	std ck	std rs
Django	379.6	516.2	48.7	1346.0	9463.4	251.3	478.7	1334.4	390.4	512.2	160.9	1383.7	2.1	21.5	14.9	4.7
Astropy	133.4	280.8	51.7	1368.3	7696.5	237.2	475.4	1490.3	383.8	532.9	176.3	1326.9	2.2	27.5	15.0	4.9
Sympy	112.2	152.7	49.6	1357.7	8579.8	249.1	531.2	1449.3	118.6	145.7	147.6	1374.4	1.6	24.2	13.8	5.9

child becomes the new active—so a later restore to this snapshot forks directly off the template instead of replaying a full CRIU restore (§4.2). *Restore* sits on the agent’s true critical path: no subsequent MCTS iteration can begin until it returns, and DeltaBox keeps it in the millisecond regime regardless of classifier tag. The record-and-replay baselines (replay+cp, replay+dk) trade cheap per-event checkpoints for reload-and-replay restores orders of magnitude larger; DeltaBox reverses this trade by hiding dump cost asynchronously and keeping the critical-path restore millisecond-scale.

copytree restore is dominated by rmtree+copytree of the testbed (80–1378 ms mean across archetypes), scaling with installed size; checkpoint pays the same one-way copy at a similar rate. docker commit checkpoint stays at ~50–73 ms across workloads because the cost is dominated by docker daemon layer-metadata commit (image-content-independent); its ~1.3 s restore pays the full container teardown + relaunch on every rollback regardless of dirty state. Firecracker Diff captures only pages dirtied since the previous snapshot at checkpoint (457–545 ms mean), but pushes the merging work to restore time: each restore must materialize a sparse copy of the base image and overlay the diffs along the MCTS ancestor chain (1.2–1.6 s mean at mean ancestor-chain depth 4–10 in 100-iter trajectories).

Restore cost on DeltaBox splits by classifier path: standard-tagged pays only the OverlayFS layer-switch concurrent with template fork() (4–6 ms mean); lightweight-parented additionally walks the symmetric ovl_lazy_read_refresh chain (+3–17 ms). The warm-template-pool interaction and slow-path fallback are discussed in §6.3.3. Firecracker Diff restore is 139–325× DeltaBox, with the prototype baselines in similar orders of magnitude (Table.2).

End-to-end, DeltaBox significantly reduces the overhead of sandbox operations within the agent’s execution flow: only the OverlayFS sink ioctl (a few ms; the std-ckpt’s CRIU dump is async and hidden under the 1–9 s LLM inference window, §4.2) plus the restore latency (5–25 ms), roughly 10–30 ms total per iteration. Baselines have no equivalent inference-masking handle and contribute their full per-event latencies directly to the critical path. Fig.6 plots the per-event CDFs from Table.2: DeltaBox’s distribution sits 1–3 orders of magnitude left of every backend across the entire range, with no long-tail crossover. These savings become decisive once LLM round-trip drops to seconds (Fig.7): at MiMo-V2.5-Pro’s

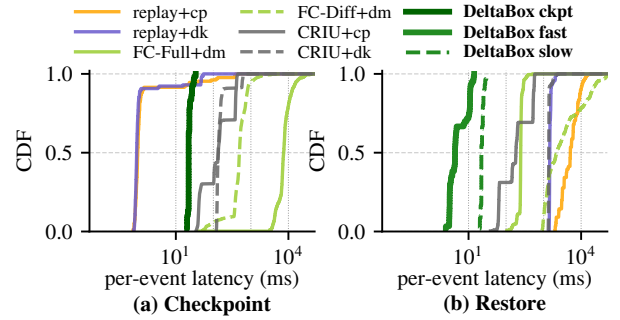


Figure 6. Per-event blocking-time CDF, pooled across the 9 MCTS trajectories of Table. 2: (a) checkpoint, (b) restore. DeltaBox’s distribution is shifted 1–3 orders of magnitude left of every coupled backend; the gap holds into the tail (no event-class crossover at any percentile).

~20 s/call, state-management overhead is a small fraction of total time, but at faster inference rates it dominates.

We replay five Qwen3-Coder-30B MCTS trajectories (median LLM RTT ~2 s, ~170 events / 100 expansions / 49 rollbacks each) end-to-end through three coupled state-management backends: DeltaBox, Firecracker Diff coupled with dm-snapshot for filesystem rollback (FC-Diff+dm), and cloud-hypervisor coupled with dm-snapshot (CHV+dm; we approximate CubeSandbox [29]’s planned event-level rollback this way since its open-source release has not yet shipped this capability). To remove any classifier-induced advantage we run DeltaBox with the LW classifier disabled (every event forced through the standard CRIU + overlay-sink path), matching the all-standard event semantics the baselines provide. Fig. 7 normalizes per-instance time to the LLM-only floor: DeltaBox stays at 1.03–1.06×, while FC-Diff+dm reaches 1.87–3.84× and CHV+dm 2.62–4.29×. State management consumes 47–77% of total time on the two coupled-FS baselines vs. 3–6% on DeltaBox.

6.2.2 Fork Primitive Throughput A motivating workload for DeltaBox is RL training, in which a single warm template spawns N parallel children per training step. We characterize the fork primitive’s scaling behavior in two layers: (i) raw kernel fork() cost on a single host (no VM, no overlays, no CRIU) to set the ceiling, and (ii) substrate-level fan-out latency including per-child filesystem isolation

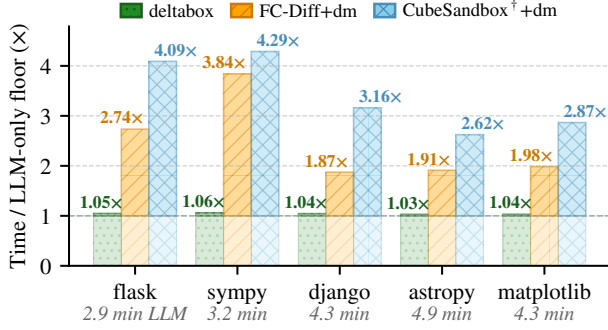


Figure 7. End-to-end time for a 100-iteration MCTS trajectory (Qwen3-Coder-30B) on five SWE-bench Verified instances, normalized per-instance to the LLM-only floor (1.0x = pure LLM RTT sum). FC-Diff+dm and CHV+dm pair each VMM with dm-snapshot for filesystem coupling; FC-Diff+dm’s chain merge follows only the MCTS ancestor path. See §6.2.1 for the CubeSandbox approximation rationale.

and process-state recovery, compared against three coupled baselines.

N	p50 (ms)	p99 (ms)	forks/s	RSS (MB)
1	0.57	0.61	1419.2	9.4
4	0.78	1.31	999.7	42.3
16	1.67	4.67	519.8	169.8
64	5.47	14.74	165.9	679.4

Table 3. Fork-out latency and footprint sweep across 18 SWE-bench MCTS trajectories (Qwen3-Coder-30B and MiMo V2.5-Pro, $\times 3$ each from Django, Sympy, Xarray archetypes; 5 reps per trajectory). Donor = DeltaBox agent .py (stdlib only) with the real trajectory loaded into the parent heap, ~ 15 MB parent RSS (LLM client decoupled via NPD, §4.2). Values are medians across the 18 trajectories; inter-trajectory IQR $< 20\%$ across all (N , metric). *forks/s*: N divided by total fan-out time. *RSS*: aggregate `/proc/<pid>/statm` resident across all N children (CoW-shared pages count toward each child).

Across the swept N , fork p50 grows sub-linearly with fan-out width (0.57 ms at $N=1$ to 5.5 ms at $N=64$); the kernel page-table copy on `fork()` is the dominant cost. Tail latency tracks p50 by $\sim 3\times$, with occasional slow-path forks at the largest fan-out (p99=14.7 ms at $N=64$). Per-child resident set is stable at ~ 11 MB across all N —children inherit the parent’s CoW-shared pages, so aggregate footprint scales linearly (9.4 $\rightarrow$ 679.4 MB across $N=1 \rightarrow 64$); a write-sensitivity pass at $N=16$ in which each child dirties a 100 MB anonymous buffer shows per-child resident rising by exactly 100 MB, confirming idle children remain predominantly CoW-shared and physical memory pressure tracks per-child write working set rather than parent template size. The fork primitive is therefore not the bottleneck for RL training fan-out at the scales we target.

Substrate-level fan-out cost. The fork-primitive sweep above (Table. 3) isolates kernel `fork()` only; production

tree-based RL fan-out additionally pays for per-child filesystem isolation and, on baselines that do not capture process state, *replay* of the K prefork agent actions to rebuild process state to the branch-point equivalent. Fig. 8(a) extends the comparison to four substrates: DeltaBox’s batched fork-from-template (Topology A, single VM with N in-VM children), copytree+replay, docker commit+replay, and Firecracker Full-snapshot restore (Topology B, N independent containers/VMs each replaying K actions).

Across the measured range DeltaBox is an order of magnitude or more faster than every Topology-B baseline (Fig. 8(a)). Against the two VM-based baselines the gap narrows at $N=64$: Firecracker’s lazy-mmap restore amortises well across N parallel processes, while CubeSandbox’s multi-step restore API (spawn, wait socket, poll `vm.info` until ready, `vm.resume`) accumulates per-instance latency under contention. The structural advantage of Topology A is that all N children share kernel pages and overlays lower layers via copy-on-write, so memory and FS-substrate cost scale sub-linearly in N .

The substrate fan-out latency in Fig. 8(a) translates directly to GPU idle time in on-policy RL training. We measure T_{gen} via vLLM [42] batched offline inference (256 $\rightarrow$ 512 tokens, $B=N$ concurrent rollouts): 1.21–3.21 s for Qwen3-4B, 1.63–3.29 s for Qwen2.5-7B-Instruct, and 3.03–6.48 s for Qwen3-14B across $N=1$ –64 (Fig. 8(b)). We additionally measure T_{train} for a Qwen2.5-7B LoRA forward+backward (rank 16, bf16, gradient-checkpointing): $B=1, 4$ on a single GPU (0.22, 0.79 s) and $B=16, 64$ on 4 GPUs with FSDP (per-GPU micro-batch 4, grad-accumulation 1 and 4): **1.14, 4.51 s** (Fig. 8(b)). Multi-GPU is required from $B=16$ upward because single-GPU activations at seq 768 already exceed 96 GB. The 4-GPU per-token cost is 0.093 ms/token-instance, $2.77\times$ faster than the single-GPU per-token cost (0.258 ms at $B=4$) thanks to DP compute parallelism — the realistic operating point for production-scale RL training.

In sync mode, $\text{step} = \text{sandbox} + T_{\text{gen}} + T_{\text{train}}$ and $\text{GPU util} = (T_{\text{gen}} + T_{\text{train}}) / \text{step}$. We validate this formula end-to-end with a 5-step full sync loop on a single GPU (real copytree+vLLM+LoRA fwd/bwd): empirical GPU-busy fraction 26.2% vs. predicted 26.5%, a 0.32 pp gap, with per-phase utilisation matching the model (0.5%/73%/99% across sandbox/gen/train). On the 7B model at $N \in \{16, 64\}$ (Fig. 8(c)) DeltaBox reaches near-saturating GPU utilization while copytree/docker sit well below; the gap widens at $N=64$ because faster multi-GPU training amplifies the relative sandbox cost.

Production frameworks (verl [43] `fully_async`, OpenRLHF [44]) close the sync gap by decoupling trainer and roll-outer at the cost of *staleness* (verl threshold < 1). With multi-GPU T_{train} , the trainer is fast enough that *all five substrates* push into rollout-bound regime at $N=16$: DeltaBox adds +0.61 stale steps, Firecracker +0.80, CubeSandbox +0.98, copytree +2.25, docker +2.11 — DeltaBox’s rollout cost is

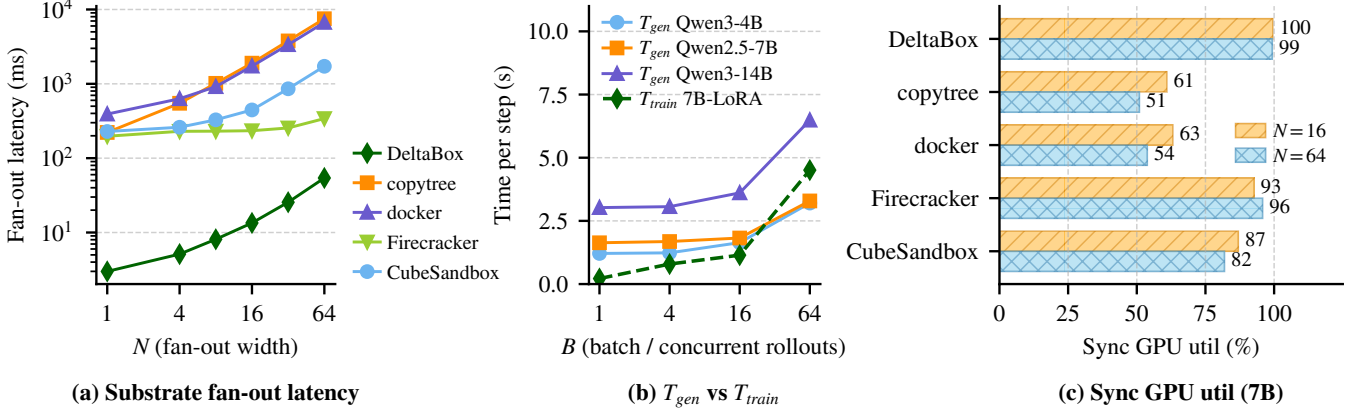


Figure 8. RL training fan-out characterisation, 5 substrates on the same GPU cluster. (a) Substrate 1:N fan-out latency on a single GPU (144 MB synthetic-anonymous parent template; $K=10$ prefork actions $\times$ 5 MB mid-state; 5 reps). The parent here is larger than DeltaBox’s realistic ~ 15 MB agent .py measured in Table.3, so this panel stress-tests the substrate primitives rather than DeltaBox’s production parent RSS. (b) T_{gen} (batched vLLM, 256 $\rightarrow$ 512 tokens, 3 reps) for three Qwen models on a single GPU, and T_{train} for a Qwen2.5-7B LoRA-r16 fwd+bwd (bf16, grad-checkpointing, 5 reps); $B \in \{1, 4\}$ on one GPU, $B \in \{16, 64\}$ on 4 GPUs (FSDP, see §6.1). (c) Sync GPU util ($(T_{\text{gen}} + T_{\text{train}}) / (\text{sandbox} + T_{\text{gen}} + T_{\text{train}})$) at $N \in \{16, 64\}$ on Qwen2.5-7B, computed inline from (a) and (b).

closest to the train budget. At $N=64$ the longer $T_{\text{train}}=4.51$ s pulls DeltaBox (+3.4 s rollout) and Firecracker (+3.6 s) back into train-bound; CubeSandbox sits marginally over (+0.11); copytree (+1.40) and docker (+1.21) clearly exceed the threshold.

6.3 Extended Studies

6.3.1 Checkpoint/Restore Latency We present the detailed latency breakdown in Table.4, which shows the cost of each component in DeltaBox in a real-world workload (SWE-bench).

Checkpoint latency. Per-step checkpoint cost is dominated by the incremental CRIU dump plus the serial template-fork tail; the DeltaFS ioctl runs concurrently and is the smaller term (Table.4).

Restore latency. On the fast path the template fork() dominates (scaling linearly with agent RSS via page-table COW) and the DeltaFS ioctl overlaps inside the fork window; the slow path is used only on first restore or after GC eviction (Table.4).

Scaling with FS dirt. Across 1–128 MB of urandom-filled (incompressible) per-event dirt (Fig. 9; controlled microbenchmark, since real swe-search edits cluster at ≤ 100 KB), DeltaBox-std checkpoint grows only mildly and DeltaBox-LW stays flat. The three semantic-parity baselines diverge by mechanism: copytree+CRIU rises at checkpoint because linear copytree dominates large sizes; FC-Diff is flat at ckpt (dominated by writing the full VM memory image regardless of dirt) and grows at restore as the Diff-chain merge lengthens monotonically; CubeSandbox writes a full VM image on every event, so it sits high on both sides. DeltaFS’s delta sink and DeltaCR’s controller-managed incremental CRIU chain (§4.2) decouple

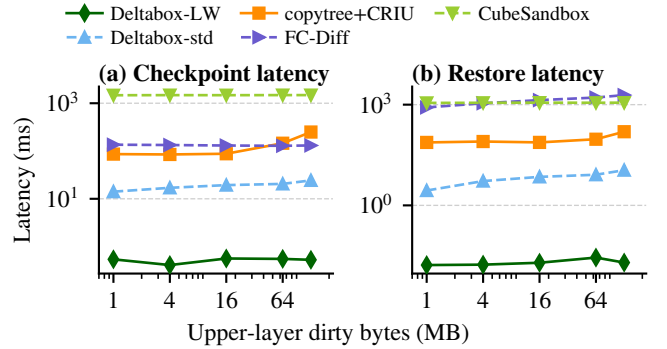


Figure 9. Per-event ckpt/restore latency vs. per-event upper-layer dirt, 5 coupled-state backends (controlled microbench; 1 warmup + 3 reps per size; 80 MB Python process; 2 GB VM for FC-Diff and CubeSandbox). All five backends capture both filesystem and process state for semantic parity with DeltaBox.

latency from both filesystem-dirt volume and accumulated chain length.

Costs of async-warm. Async-warm operates on both restore paths: on the common fast path it pre-pays CoW faults after the template fork(), and on the slow path it overlaps with CRIU’s userfaultfd page-in from the tmpfs dump images. We characterise the fast path below, since it is the dominant case; the slow path inherits the same idle-window absorption argument. DeltaBox’s fork-based restore is fast precisely because it duplicates only page tables: the new child runs immediately on pages still shared with the template, with no physical memory copy on the critical path. The flip side is that every page is CoW-shared, so without intervention the agent’s first write to each hot page triggers a synchronous CoW fault on the critical path, accumulating

Table 4. DeltaBox component latency breakdown, mean across SWE-bench Astropy/Django/Sympy (per-workload variance $\leq 4\%$). Fast path is the common case; slow path runs on template eviction.

Component	ck	rs (fast)	rs (slow)
DeltaFS layer switch (ioctl)	1.12	1.66 [†]	1.66 [†]
DeltaCR incr. dump + fork	13.45	—	—
DeltaCR template fork (alone)	—	3.75 [†]	—
DeltaCR CRIU restore (alone)	—	—	7.25 [†]
Hot-page async-warm	—	async [‡]	async [‡]
Agent-perceived blocking	0	5.14[§]	8.04[§]

[†]ioctl runs concurrently with fork/CRIU; agent-perceived blocking = max of in-parallel components. [‡]Asynchronous page-warming, not perceived by the agent: on the fast path a GSD thread pre-touches hot zones to absorb CoW faults after fork(); on the slow path CRIU’s lazy-pages daemon serves remaining faults concurrently with the resumed agent (Fig. 10, §4.2). [§]Restore total exceeds max of in-parallel components by 0.5–2.2 ms due to scheduler + dispatch overhead. ^{||}During ck the agent is already blocked on LLM inference; the sandbox’s 14.57 ms of local OS work (IOCTL + INCR. DUMP + FORK) overlaps that window via the network proxy daemon (§4.2) and contributes zero blocking that the agent perceives beyond the LLM wait.

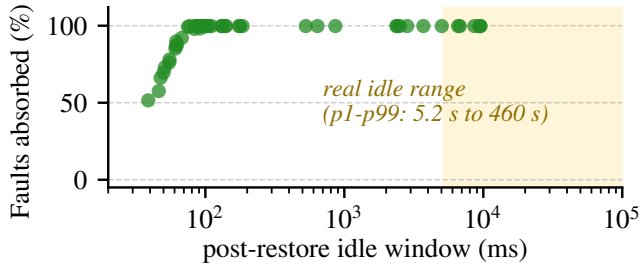


Figure 10. Per-event CoW fault absorption vs. post-restore idle window (50 swe-search MCTS restore events). Shaded band: p1–p99 idle range from 2,311 LLM-driven restore events.

hundreds-of-microseconds latencies across the post-restore turn. Async-warm pre-pays these faults from a GSD daemon thread off the critical path, so the agent’s later writes find their pages already private. Fig. 10 verifies that lazy-CoW restore does not accumulate post-fork page-fault debt under realistic LLM idle windows.

Comparison. DeltaBox’s fast-path restore is the only millisecond-scale point across the coupled-backend comparison (Table. 4, Table. 2): it dumps only the agent’s private pages rather than the whole VM image, and avoids a container restart on rollback.

6.3.2 Write Amplification We measure per-edit *copy-up bytes* (file data re-materialized into the upper layer) and *physical I/O bytes* (loopback sectors written, including journal and metadata) over real swe-search agent edits sized 1–256 KB, across ext4, XFS, and XFS+reflink.

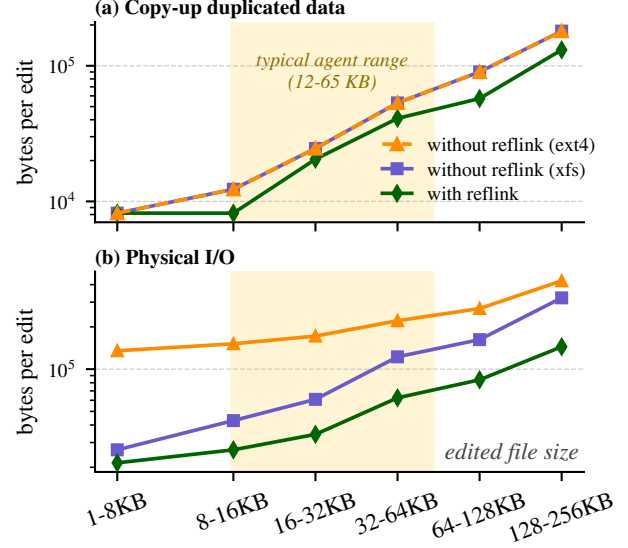


Figure 11. Per-edit copy-up bytes (a) and physical I/O bytes (b) vs. edited-file size (log–log); real SWE-bench agent edits across three filesystem configurations, per-bin medians. Shaded band marks the typical agent edit range. ext4 and XFS-without-reflink coincide on (a): copy-up benefit comes entirely from reflink, not XFS.

Reflink savings on copy-up grow with file size; XFS without reflink offers no advantage. Fig. 11(a) shows ext4 and XFS-without-reflink coinciding at every bin: both perform a full file recopy when an existing file is modified, so per-edit copy-up grows linearly with file size. Reflink-aware copy-up shares extents with the lower layer; only the 4 KB blocks the partial-write actually dirties contribute to duplicated bytes, so the reflink curve sits below the no-reflink lines but tracks them in slope. The benefit grows with file size more modestly than the asymptotic block-sharing model would predict, because real edits rarely land near the file’s end where the reflinked prefix would be largest.

Physical I/O reduction comes from both XFS metadata efficiency and reflink. Fig. 11(b) shows that XFS’s lighter metadata bookkeeping dominates on small files (132 KB $\rightarrow$ 26 KB at 1–8 KB, with reflink adding little), while reflink’s block sharing dominates on large files (315 KB $\rightarrow$ 141 KB at 128–256 KB). The two mechanisms are complementary across the edit-size range.

6.3.3 Adaptive Optimization Effectiveness

Lightweight skip ratio. Across 87 production MCTS runs spanning 9 SWE-bench Verified repositories (1,689 checkpoint events total), the semantic classifier elides entire checkpoints whose agent action neither mutates process memory nor writes to the upper layer. The pooled weighted-mean skip ratio across all 9 repositories is **62.0%**. The primary benefit of LW elision is reduced RAM pressure on the in-VM tmpfs snapshot store rather than checkpoint latency,

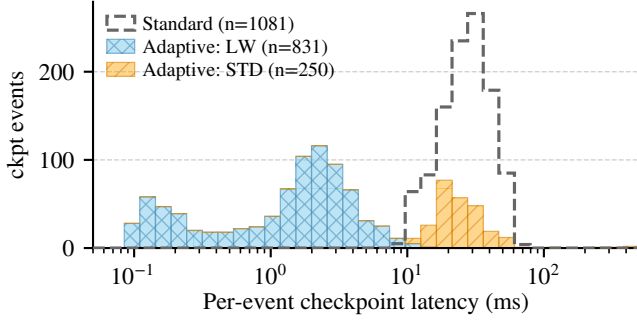


Figure 12. Per-event checkpoint latency on 1,689 ckpt events from 87 MCTS runs across 9 SWE-bench Verified repositories. *Adaptive*: pure-read cmds (*LW*, blue; $n=1047$) skip the dump; FS-mutating cmds (*std*, orange; $n=642$) take the full incremental dump. *Standard* (gray dashed): same events forced through the std path; 62.0% of events route to the LW peak.

since the CRIU dump on the standard path is asynchronous and masked by LLM inference (§6.2.1); when an event *does* block on the agent’s critical path, LW saves only the ms-scale ioctl-skip headroom captured by Table 2.

GC effectiveness. In a stress test with moderate branching factor and depth, the GC mechanism reclaims snapshot storage at each prune event. GC runs asynchronously and adds no observable latency to the active checkpoint path. Without GC, cumulative storage grows monotonically; with GC, storage remains bounded and proportional to the number of live branches.

6.3.4 Cumulative Storage Overhead For workloads with GB-scale cached files, the combination of lightweight skip and XFS reflink is designed to reduce cumulative snapshot footprint compared to full file-copy baselines; Fig. 11(a) quantifies the per-snapshot copy-up saving at the 100 KB per-file scale and (b) shows the resulting on-device physical-write reduction. Across nine SWE-bench MCTS trajectories replayed through real *criu* dump on a Python process sized to the agent.py controller, reachability-aware GC (§ 5.2.1) reduces DeltaBox’s end-of-trajectory dump storage by 46–63% versus retaining every checkpoint (Fig. 13).

7 Related Work

Agent sandboxes. Recent agent sandboxes [4, 29, 45–47] primarily optimize isolation and startup latency, but provide only coarse-grained rollback. Daytona’s OCI workspaces [45] use Docker-layer commits and miss live process state. E2B [4] uses Firecracker microVMs and pre-warmed templates, improving isolation but retaining VM-granularity snapshot costs. ZeroBoot [46] pushes VM cloning further with KVM-level CoW memory, but each branch remains an entire VM, with a shared block device, kernel-level dirty state, and no independent rollback for

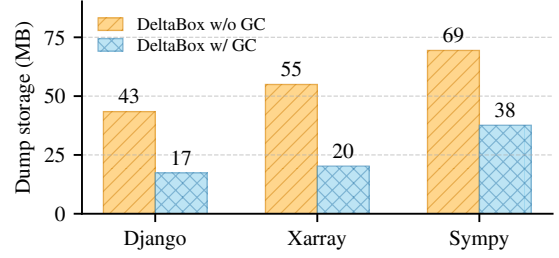


Figure 13. End-of-trajectory CRIU dump storage on 9 SWE-bench instances (3 per archetype, averaged), replayed through real *criu* dump on a 5 MB Python process matched to the Mode A footprint. Comparison: reachability-aware GC (§5.2.1) versus retaining every checkpoint.

multiple in-VM trajectories. Yan’s fault-tolerant sandbox [47] moves toward transactional agent execution, but its filesystem rollback based on tool interception and userspace CoW simulation yields second-scale checkpoints and no coupled memory snapshot. DeltaBox keeps the sandbox abstraction lightweight enough for many branches inside one VM, while providing tens-of-milliseconds, coupled filesystem-memory checkpoint/restore for stateful agent search and training workloads.

Sandboxes for serverless computing. Serverless systems have long optimized sandbox reuse, cold start, and snapshot restore, but their unit of reuse is a function invocation rather than a rollback point inside a long-lived agent trajectory. TrEnv-X [48] repurposes sandboxes across invocations using OS-level memory templates backed by CXL or RDMA memory pools, while earlier cold-start systems accelerate process creation, language runtime initialization, or post-restore fault handling for short-lived functions [15–17, 35, 36, 49]; Spice [49] further moves restore-path fault resolution into the kernel with Overlay VMAs. These techniques are orthogonal to DeltaBox. A serverless instance typically restores a compute image and then runs one request, whereas DeltaBox repeatedly descends and backtracks within one task.

Checkpointing optimizations. Existing checkpointing systems [50–55] provide important building blocks, but none provide the coupled, fine-grained rollback interface needed by stateful agents. CRIU [26] enables process-level checkpoint/restore and underlies DeltaCR’s incremental dump path, while DMTCP [56] targets transparent distributed checkpointing rather than Linux soft-dirty, high-frequency agent snapshots. Firecracker [27] and Sabre [37] restore whole VMs, which is too coarse for branches isolated inside one VM. Filesystem snapshots in Btrfs [57] and ZFS [58] provide block-level CoW but require different storage stacks, and EROFS [59] supplies read-only overlay layers without replacing the writable upperdir. DeltaFS instead hot-switches overlayfs layers over XFS reflinks. At the application layer, LangGraph [38] and LangChain [41] checkpoint logical

agent state, which cannot undo the effects of executed commands.

8 Conclusion

We present DeltaBox, an OS-level rollbackable sandbox designed to accelerate AI agent workloads such as test-time tree search and reinforcement learning. Recognizing that subsequent agent states are highly similar, DeltaBox eschews full state duplication in favor of diff-based checkpoint and restore. To achieve this, DeltaFS enables dynamic, unmount-free overlays layer switching for filesystem C/R, while DeltaCR utilizes incremental dumps and warm-template forking for memory C/R.

References

- [1] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 54107–54157. https://proceedings.iclr.cc/paper_files/paper/2024/file/edac78c3e300629acfe6cbe9ca88fb84-Paper-Conference.pdf
- [2] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 15585–15606. https://proceedings.iclr.cc/paper_files/paper/2024/file/4410c0711e9154a7a2d26f9b3816d1ef-Paper-Conference.pdf
- [3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] <https://arxiv.org/abs/2210.03629>
- [4] E2B. 2024. E2B: The Enterprise AI Agent Cloud. <https://e2b.dev>.
- [5] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. 10131–10165. https://proceedings.iclr.cc/paper_files/paper/2025/file/1b623663fd9b874366f3ce019dfdd44-Paper-Conference.pdf
- [6] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML ’24)*. JMLR.org, Article 2572, 23 pages.
- [7] Cheng Zhang, Erhu Feng, Xi Zhao, Yisheng Zhao, Wangbo Gong, Jiahui Sun, Dong Du, Zhichao Hua, Yubin Xia, and Haibo Chen. 2025. MobiAgent: A Systematic Framework for Customizable Mobile Agents. arXiv:2509.00531 [cs.MA] <https://arxiv.org/abs/2509.00531>
- [8] The OpenClaw Project. 2026. openclaw/openclaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way. <https://github.com/openclaw/openclaw>.
- [9] OpenAI. 2024. OpenAI o1 System Card. arXiv:2412.16720 [cs.AI] <https://arxiv.org/abs/2412.16720>
- [10] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.AI] <https://arxiv.org/abs/2501.12948>
- [11] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787 [cs.LG] <https://arxiv.org/abs/2407.21787>
- [12] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. 2026. History Doesn’t Repeat Itself but Roll-outs Rhyme: Accelerating Reinforcement Learning with RhymeRL. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (USA) (ASPLOS ’26)*. Association for Computing Machinery, New York, NY, USA, 929–945. <https://doi.org/10.1145/3779212.3790172>
- [13] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] <https://arxiv.org/abs/2402.03300>
- [14] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, juncai liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, and Mingxuan Wang. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. In *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (Eds.), Vol. 38. Curran Associates, Inc., 113222–113244. https://proceedings.neurips.cc/paper_files/paper/2025/file/a4277440d50f1f15d2cb4c14f7e0c0d2-Paper-Conference.pdf
- [15] Lixiang Ao, George Porter, and Geoffrey M. Voelker. 2022. FaaSnap: FaaS made fast using snapshot-based VMs. In *Proceedings of the Seventeenth European Conference on Computer Systems (Rennes, France) (EuroSys ’22)*. Association for Computing Machinery, New York, NY, USA, 730–746. <https://doi.org/10.1145/3492321.3524270>
- [16] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. 2020. Catalyzer: Sub-millisecond Startup for Serverless Computing with Initialization-less Booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS ’20)*. Association for Computing Machinery, New York, NY, USA, 467–481. <https://doi.org/10.1145/3373376.3378512>
- [17] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. 2021. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Virtual, USA) (ASPLOS ’21)*. Association for Computing Machinery, New York, NY, USA, 559–572. <https://doi.org/10.1145/3445814.3446714>
- [18] Xiaohu Chai, Tianyu Zhou, Keyang Hu, Jianfeng Tan, Tiwei Bie, Anqi Shen, Dawei Shen, Qi Xing, Shun Song, Tongkai Yang, Le Gao, Feng Yu, Zhengyu He, Dong Du, Yubin Xia, Kang Chen, and Yu Chen. 2025. Fork in the road: reflections and optimizations for cold start latency in production serverless systems. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation (Boston, MA, USA) (OSDI ’25)*. USENIX Association, USA, Article 28, 20 pages.
- [19] Lazar Cvetković, François Costa, Mihajlo Djokic, Michal Friedman, and Ana Klimovic. 2024. Dirigent: Lightweight Serverless Orchestration. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (Austin, TX, USA) (SOSP ’24)*. Association for Computing Machinery, New York, NY, USA, 369–384. <https://doi.org/10.1145/3694715.3695966>
- [20] Dong Du, Qingyuan Liu, Xueqiang Jiang, Yubin Xia, Binyu Zang, and Haibo Chen. 2022. Serverless computing on heterogeneous

- computers. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '22). Association for Computing Machinery, New York, NY, USA, 797–813. <https://doi.org/10.1145/3503222.3507732>
- [21] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. 2022. RunD: A Lightweight Secure Container Runtime for High-density Deployment and High-concurrency Startup in Serverless Computing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 53–68. <https://www.usenix.org/conference/atc22/presentation/li-zijun-rund>
 - [22] Hanfei Yu, Rohan Basu Roy, Christian Fontenot, Devesh Tiwari, Jian Li, Hong Zhang, Hao Wang, and Seung-Jong Park. 2024. Rainbow-Cake: Mitigating Cold-starts in Serverless with Layer-wise Container Caching and Sharing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 335–350. <https://doi.org/10.1145/3617232.3624871>
 - [23] Jialiang Huang, MingXing Zhang, Teng Ma, Zheng Liu, Sixing Lin, Kang Chen, Jinlei Jiang, Xia Liao, Yingdi Shan, Ning Zhang, Mengting Lu, Tao Ma, Haifeng Gong, and Yongwei Wu. 2024. TrEnv: Transparently Share Serverless Execution Environments Across Different Functions and Nodes. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 421–437. <https://doi.org/10.1145/3694715.3695967>
 - [24] Ariel Szekely, Adam Belay, Robert Morris, and M. Frans Kaashoek. 2024. Unifying serverless and microservice workloads with SigmaOS. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 385–402. <https://doi.org/10.1145/3694715.3695947>
 - [25] E2B. 2026. E2B Sandbox persistence. <https://e2b.dev/docs/sandbox/persistence>.
 - [26] The CRIU Project. 2011. CRIU: Checkpoint/Restore In Userspace. <https://criu.org>.
 - [27] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana Maria Popa. 2020. Firecracker: lightweight virtualization for serverless applications. In *Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation* (Santa Clara, CA, USA) (NSDI'20). USENIX Association, USA, 419–434.
 - [28] DeepSeek-AI. 2026. *DeepSeek-V4 Technical Report*. Technical Report. DeepSeek-AI. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf
 - [29] Tencent Cloud. 2026. CubeSandbox. <https://github.com/TencentCloud/CubeSandbox>.
 - [30] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 52040–52094. <https://doi.org/10.52202/079017-1650>
 - [31] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 52989–53046. https://proceedings.iclr.cc/paper_files/paper/2024/file/e9df36b21ff4ee211a8b71ee8b7e9f57-Paper-Conference.pdf
 - [32] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 50528–50652. <https://doi.org/10.52202/079017-1601>
 - [33] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. 65882–65919. https://proceedings.iclr.cc/paper_files/paper/2025/file/a4b6ad6b48850c0c331d1259fc66a69c-Paper-Conference.pdf
 - [34] Paul Gauthier. 2023. Aider: AI Pair Programming in Your Terminal. <https://aider.chat/>.
 - [35] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *2018 USENIX Annual Technical Conference (ATC)*. USENIX Association, 57–70. <https://www.usenix.org/conference/atc18/presentation/oakes>
 - [36] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. 2020. SEUSS: skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems* (Heraklion, Greece) (EuroSys '20). Association for Computing Machinery, New York, NY, USA, Article 32, 15 pages. <https://doi.org/10.1145/3342195.3392698>
 - [37] Nikita Lazarev, Varun Gohil, James Tsai, Andy Anderson, Bhushan Chitlur, Zhiru Zhang, and Christina Delimitrou. 2024. Sabre: hardware-accelerated snapshot compression for serverless MicroVMs. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (OSDI'24). USENIX Association, USA, Article 1, 18 pages.
 - [38] LangChain, Inc. 2024. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. <https://github.com/langchain-ai/langgraph>.
 - [39] Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. 2025. Agent Lightning: Train ANY AI Agents with Reinforcement Learning. arXiv:2508.03680 [cs.AI] <https://arxiv.org/abs/2508.03680>
 - [40] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. 2025. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. 64485–64515. https://proceedings.iclr.cc/paper_files/paper/2025/file/a1e6783e4d739196cad3336f12d402bf-Paper-Conference.pdf
 - [41] LangChain, Inc. 2022. LangChain: Building Applications with LLMs through Composability. <https://github.com/langchain-ai/langchain>.
 - [42] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP '23)*.
 - [43] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*.

- [44] Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. 2024. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. arXiv:2405.11143 [cs.AI] <https://arxiv.org/abs/2405.11143>
- [45] Daytona. 2024. Daytona. <https://daytona.io>.
- [46] ZeroBoot. 2026. ZeroBoot: Sub-millisecond VM Sandboxes for AI Agents via Copy-on-Write Forking. <https://github.com/zerobootdev/zeroboot>.
- [47] Boyang Yan. 2025. Fault-Tolerant Sandboxing for AI Coding Agents: A Transactional Approach to Safe Autonomous Execution. arXiv:2512.12806 [cs.AI] <https://arxiv.org/abs/2512.12806>
- [48] Jialiang Huang, Teng Ma, Zheng Liu, Sixing Lin, Kang Chen, Jinlei Jiang, Xia Liao, Yingdi Shan, Yongwei Wu, Ning Zhang, Mengting Lu, Tao Ma, Haifeng Gong, and Mingxing Zhang. 2026. TrEnv-X: Transparently Share Serverless Execution Environments Across Different Functions and Nodes. *ACM Transactions on Computer Systems* (March 2026). <https://doi.org/10.1145/3805475>
- [49] Ben Holmes, Baltasar Dinis, Lana Honcharuk, Joshua Fried, and Adam Belay. 2025. Taming Serverless Cold Starts Through OS Co-Design. arXiv:2509.14292 [cs.OS] <https://arxiv.org/abs/2509.14292>
- [50] Yanning Yang, Dong Du, Haitao Song, and Yubin Xia. 2024. On-demand and Parallel Checkpoint/Restore for GPU Applications. In *Proceedings of the 2024 ACM Symposium on Cloud Computing* (Redmond, WA, USA) (SoCC '24). Association for Computing Machinery, New York, NY, USA, 415–433. <https://doi.org/10.1145/3698038.3698510>
- [51] P. Tullmann, J. Lepreau, B. Ford, and M. Hibler. 1996. User-level checkpointing through exportable kernel state. In *Proceedings of the Fifth International Workshop on Object-Oriented in Operation Systems*. 85–88. <https://doi.org/10.1109/IWOOS.1996.557874>
- [52] Dirk Vogt, Armando Miraglia, Georgios Portokalidis, Herbert Bos, Andy Tanenbaum, and Cristiano Giuffrida. 2015. Speculative Memory Checkpointing. In *Proceedings of the 16th Annual Middleware Conference* (Vancouver, BC, Canada) (Middleware '15). Association for Computing Machinery, New York, NY, USA, 197–209. <https://doi.org/10.1145/2814576.2814802>
- [53] A. Dearle and D. Hulse. 1995. On page-based optimistic process checkpointing. In *Proceedings of International Workshop on Object Orientation in Operating Systems*. 24–32. <https://doi.org/10.1109/IWOOS.1995.470583>
- [54] Emil Tsalapatis, Ryan Hancock, Tavian Barnes, and Ali José Mashitizadeh. 2021. The Aurora Single Level Store Operating System. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) (SOSP '21). Association for Computing Machinery, New York, NY, USA, 788–803. <https://doi.org/10.1145/3477132.3483563>
- [55] James S. Plank, Micah Beck, Gerry Kingsley, and Kai Li. 1995. Libckpt: transparent checkpointing under Unix. In *Proceedings of the USENIX 1995 Technical Conference Proceedings* (New Orleans, Louisiana) (TCO'95). USENIX Association, USA, 18.
- [56] Jason Ansel, Kapil Arya, and Gene Cooperman. 2009. DMTCP: Transparent checkpointing for cluster computations and the desktop. In *Proceedings of the 2009 IEEE International Symposium on Parallel and Distributed Processing (IPDPS '09)*. IEEE Computer Society, USA, 1–12. <https://doi.org/10.1109/IPDPS.2009.5161063>
- [57] The Btrfs Project. 2009. Btrfs Documentation. <https://btrfs.readthedocs.io>.
- [58] OpenZFS. 2013. OpenZFS Documentation. <https://openzfs.github.io/openzfs-docs/>.
- [59] Linux Kernel Project. 2018. EROFS: Enhanced Read-Only File System. <https://erofs.docs.kernel.org>.